IPC

SK UNIVERSITY

Everything is a file

- **Actually, "*Everything is a file descriptor*"**

- **Pros**
  - Can reuse tools, APIs on a wide range of resources
- **Cons**
  - Not a fast or portable approach

- **Communication using file interface?**

IPC

- **Inter-Process Communication**
  - Mechanism for **communication between processes**

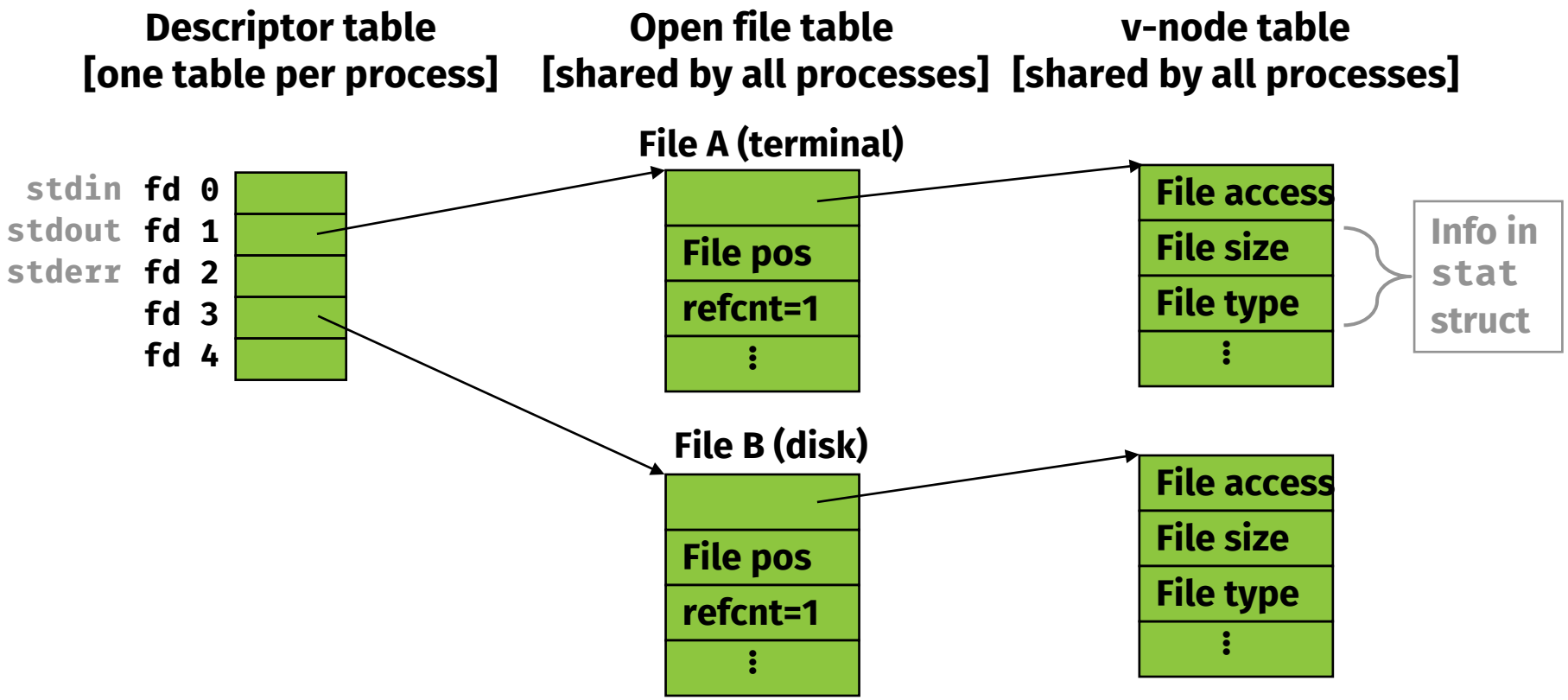
- **Methods in IPC**
  - Signals
  - **I/O redirection**
  - **Anonymous Pipe**
  - **Named Pipe (FIFO)**
  - Shared Memory, Message Queue, etc.

Open Files in Kernel

- **How does the Unix kernel represent open files?**
- **3-levels**
  - Descriptor table
    - » 1 table **per process**
    - » Pointer to file entry in the "open file table"
  - Open File table
    - » Shared by **all processes**
    - » Current file position, mode, reference count, pointer to file entry in the "v-node table"
  - v-node table
    - » Shared by **all processes**
    - » Information about file itself (size, permission, type, etc.)

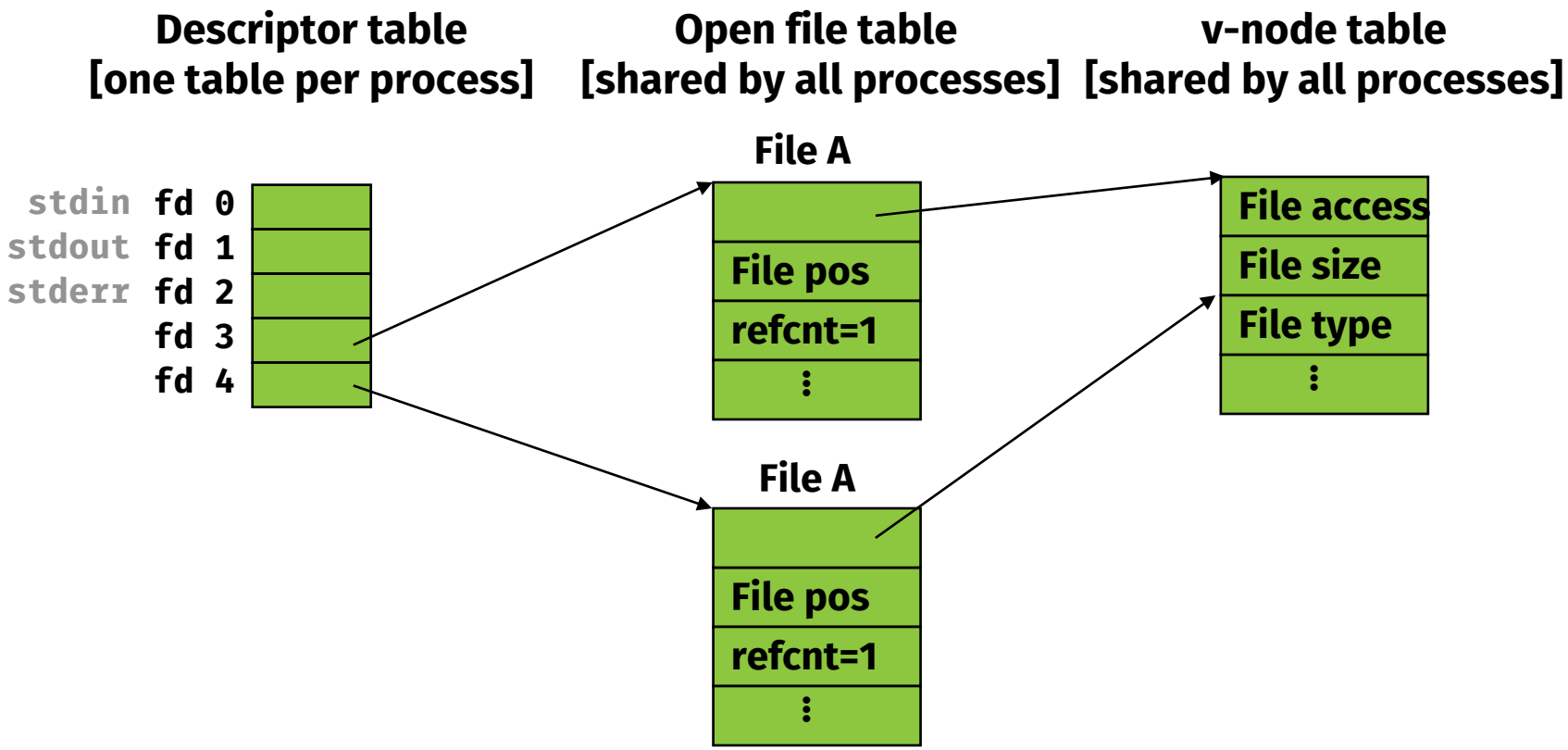
Open Files in Kernel (2)

- How does the Unix kernel represent open files?



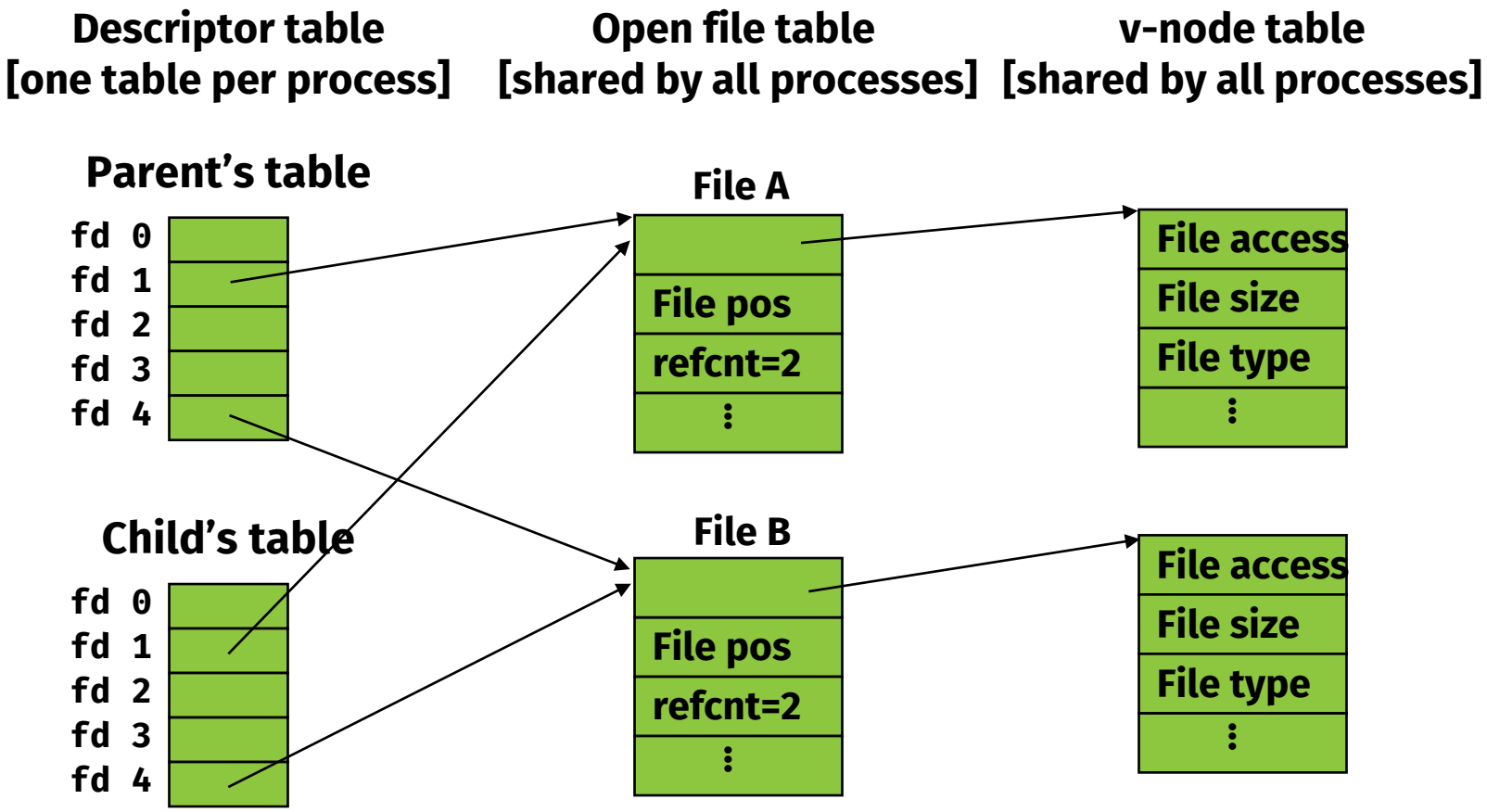
Open Files in Kernel (3)

- Calling *open( )* twice with the same *filename*



Open Files in Kernel (4)

- Calling *fork( )*



Open Files in Kernel (5)

- **What will be the result?**

```c
int main(void) {
    char c;
    int fd = open("temp.txt", O_RDONLY);

    switch (fork()) {
        case -1:
            perror("fork failed");
            exit(1);
        case 0:
            assert(read(fd, &c, 1) >= 0);
            exit(0);
        default:
            wait(NULL);
            assert(read(fd, &c, 1) >= 0);
            printf("c=%c\n",c);
    }

    return 0;
}
```

```c
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <assert.h>
#include <stdio.h>
```

temp.txt

sysexperiment

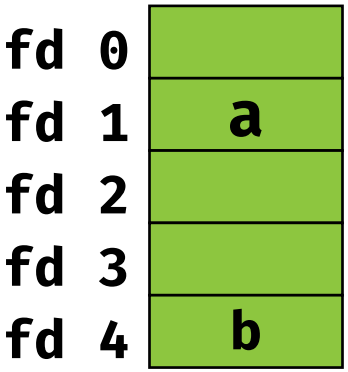
I/O Redirection

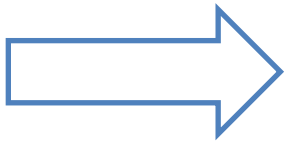
- **Q: How does a *shell* implement I/O redirection?**

```
$ ls > foo.txt
```

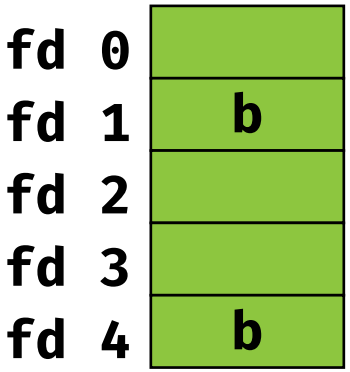
- **A: By calling the `dup2(oldfd, newfd)` function**
  - Copies (per-process) descriptor table entry **`oldfd`** to entry **`newfd`**

Descriptor table before `dup2(4, 1)`



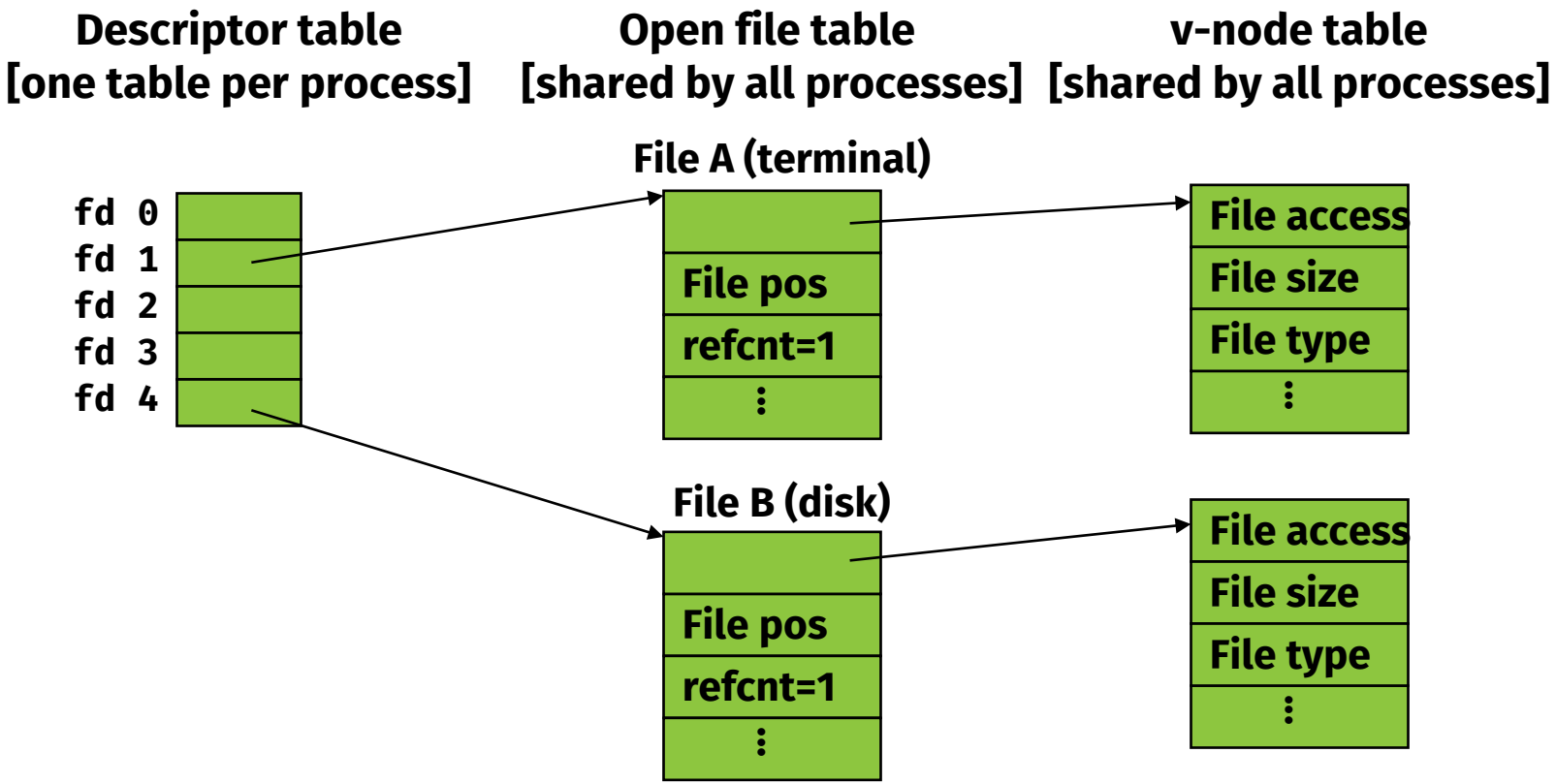


Descriptor table after `dup2(4, 1)`



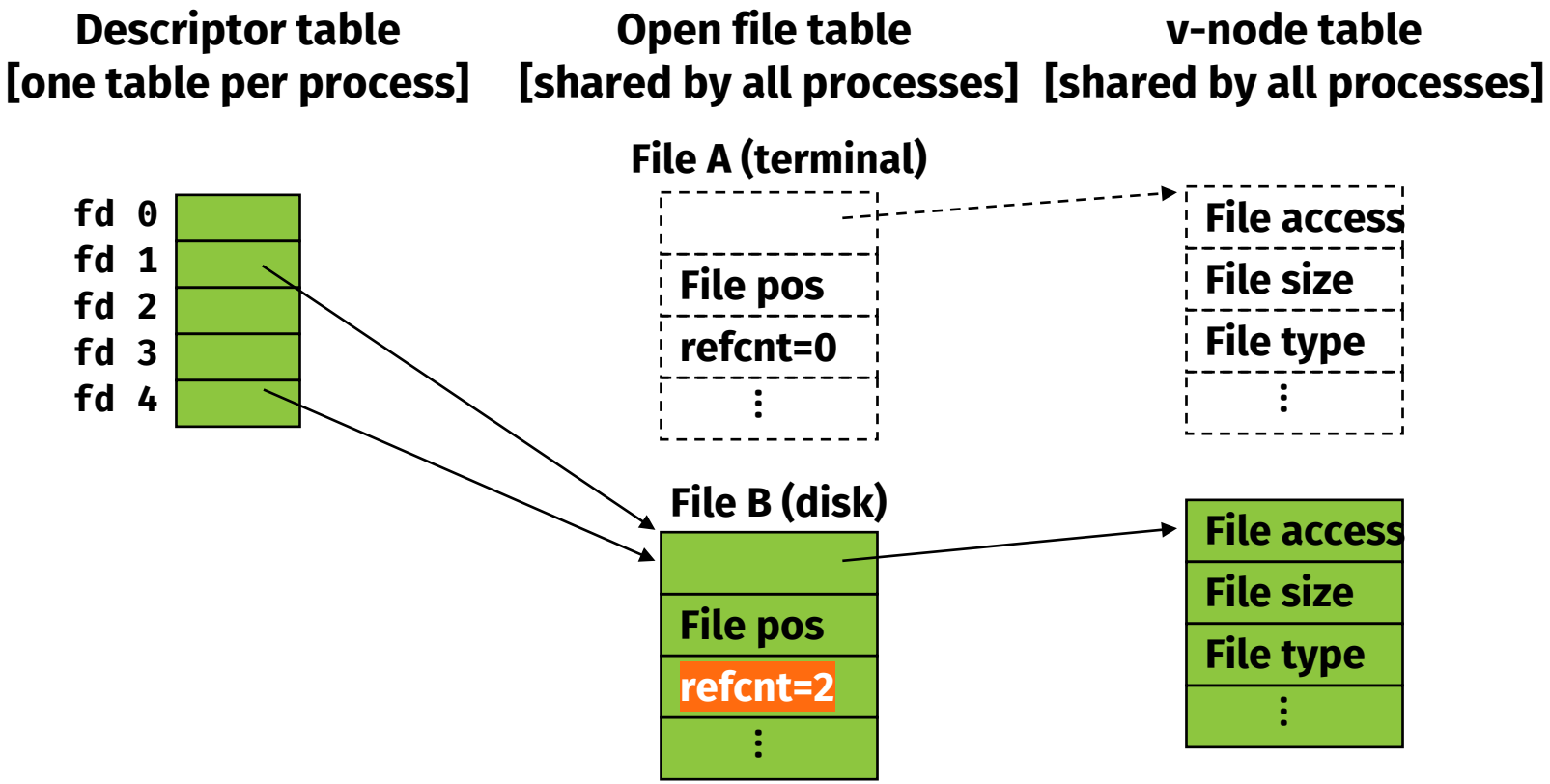
I/O Redirection Example (1)

- Before calling `dup2(4,1)`



I/O Redirection Example (2)

- After calling `dup2(4,1)`

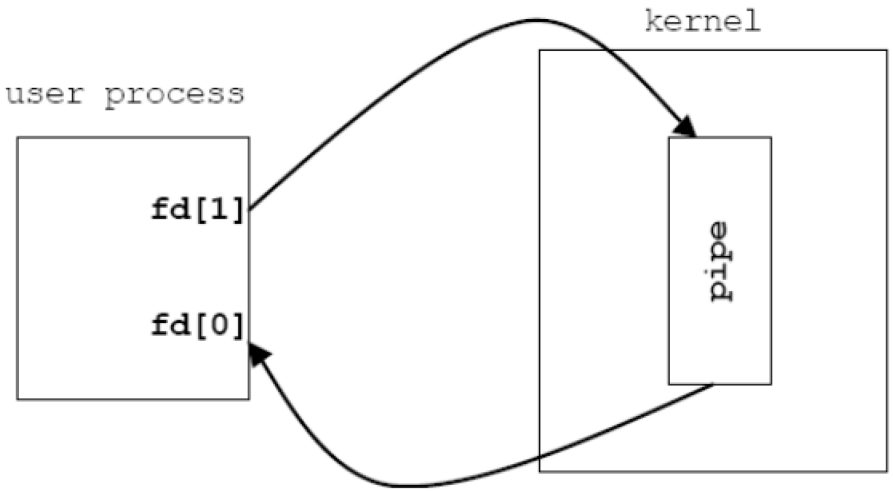


Pipes

- **Pipes**
  - Oldest mechanism for IPC in UNIX
  - Communicate with shared file descriptor of pipe
- **Limitations**
  - Half-duplex: data flows only in one direction
  - Data only can be read once
- **Two pipes**
  - Anonymous pipe
    - » Temporary, between parent-child
  - Named pipe (FIFO)
    - » Saved in the file system, between arbitrary processes

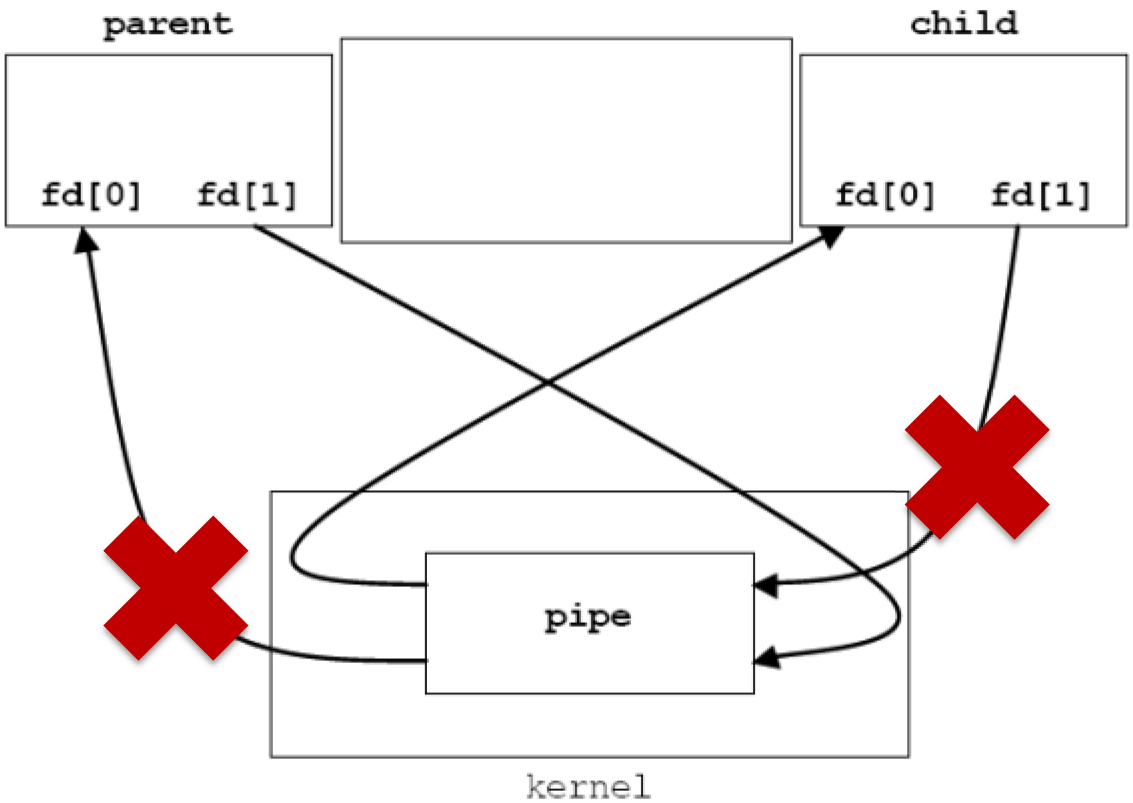
Anonymous Pipe (1)

- `int pipe(int fd[2]);`
  - Two file descriptors are returned through the **`fd`** argument
    - » **`fd[0]`**: open for reading
    - » **`fd[1]`**: open for writing
  - The output of **`fd[1]`** is the input for **`fd[0]`**
  - Return **`-1`** on error



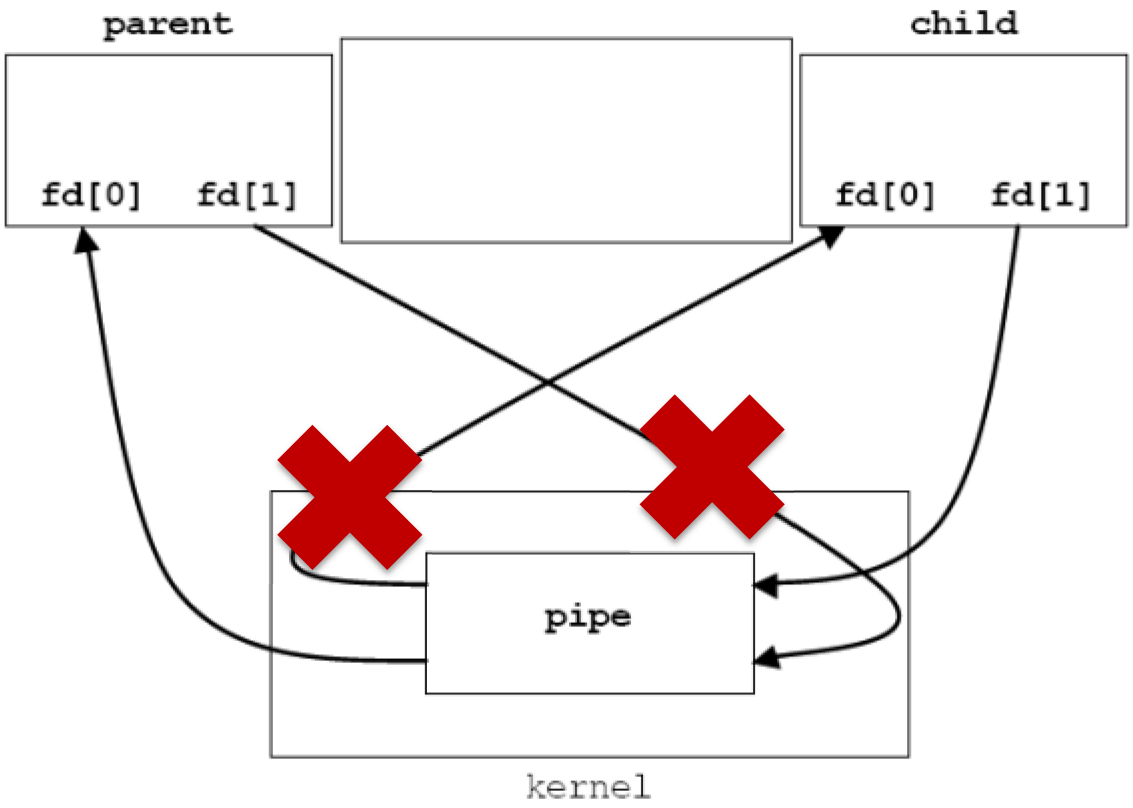
Anonymous Pipe (2)

parent → child:
Parent closes fd[0];
Child closes fd[1];



Anonymous Pipe (3)

parent ← child:
Parent closes fd[1];
Child closes fd[0];



Using Anonymous Pipe

```c
#include <unistd.h>
#include <stdlib.h>

#define MAXLINE 80

int main(void) {
    int n, fd[2];
    char line[MAXLINE];

    if(pipe(fd) < 0) exit(1);

    switch (fork()) {
        case -1:
            exit(2);
        case 0: /* child */
            close(fd[1]);
            n = read(fd[0], line, MAXLINE);
            write(1, line, n);
        default: /* parent */
            close(fd[0]);
            write(fd[1], "hi!\n", 12);
    }
    _exit(0);
}
```

Named Pipe (FIFO)

- `int mknod(const char* path, mode_t mode, dev_t dev);`
  - `mknod("pathname", S_IFIFO | SIRUSR | S_IWUSR, 0);`

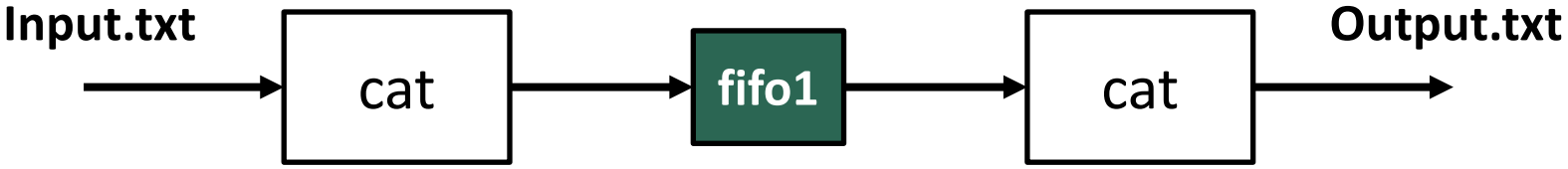
- `int mkfifo(const char* path, mode_t mode);`
  - `mkfifo("pathname", SIRUSR | S_IWUSR);`

- **You can also make FIFOs on the command line by using "`mknod`" and "`mkfifo`"**
- **open() call with read-/write-only mode**
  - Call will be blocked until another process opens file for writing/reading.
  - Can be bypassed with **`O_NONBLOCK`**.

Use of FIFOs (1)

- **Duplicating a Stream**
  - Passing data from one shell pipeline to another without creating intermediate temporary files

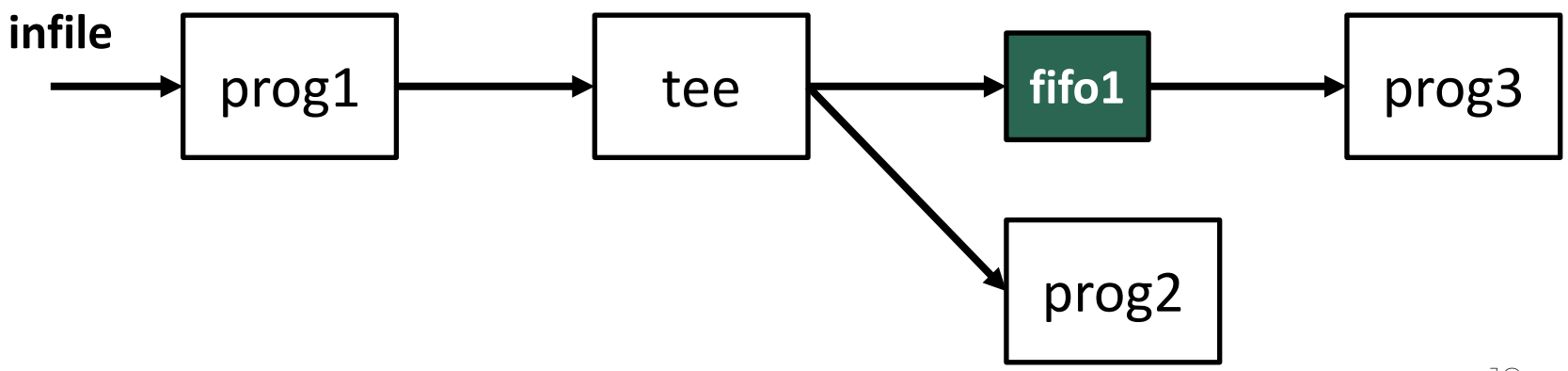
```
$ mkfifo fifo1
$ cat < fifo1 > output.txt &
$ cat < input.txt > fifo1
```



Use of FIFOs (1)

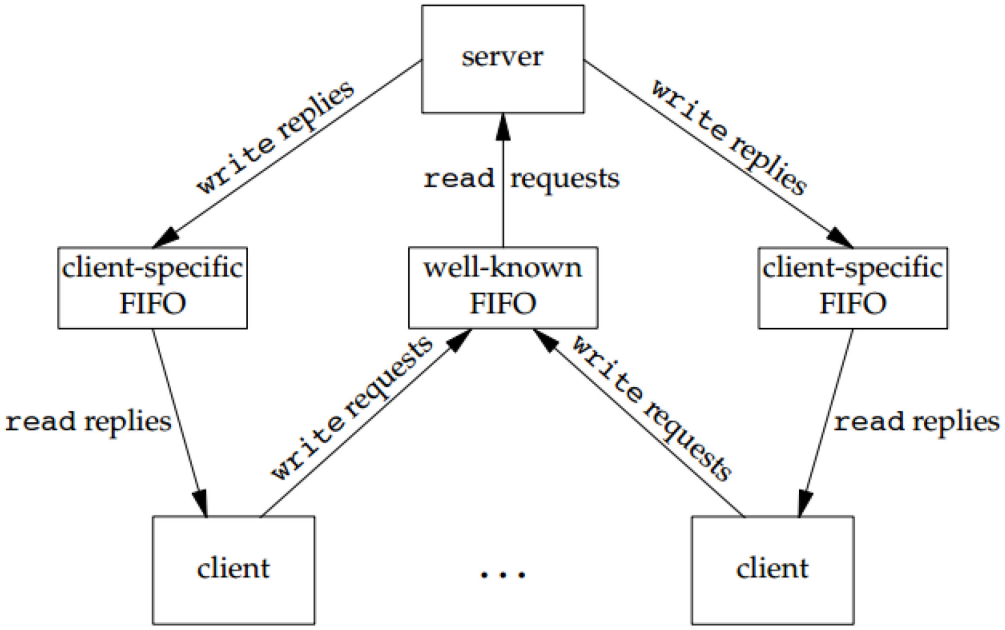
- **Duplicating a Stream**
  - Passing data from one shell pipeline to another without creating intermediate temporary files

```
$ mkfifo fifo1
$ prog3 < fifo1 &
$ prog1 < infile | tee fifo1 | prog2
```



Use of FIFOs (2)

- **Client-server Communication**
  - A client-server application to pass data between the client and server **on the same machine**
    - » Clients write to a “well-known” FIFO to send a request to the server



Using Named Pipe (Server)

```c
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

#define FIFO_FILE "fifo_file"
#define MAX_SIZE 100

// In actual applications, this must be (de-)serialized
typedef struct message {
  char username[MAX_SIZE];
  char message[MAX_SIZE];
  struct tm timestamp;
} Message;
```

```c
int main() {
  Message msg;

  mkfifo(FIFO_FILE, S_IRUSR | S_IWUSR |
                    S_IRGRP | S_IWGRP |
                    S_IROTH | S_IWOTH);
  printf("Server waiting for client...\n");

  int fd = open(FIFO_FILE, O_RDONLY);
  printf("Client connected! \n");

  while (read(fd, (char *)&msg, sizeof(Message)) > 0) {
    char timestamp[20];
    strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S", &msg.timestamp);
    printf("[%s] %s: %s\n", timestamp, msg.username, msg.message);
  }
  printf("Connection terminated...\nGoodbye!\n");
  close(fd);
  unlink(FIFO_FILE);

  return 0;
}
```

Using Named Pipe (Client)

```c
int main() {
  int fd = open(FIFO_FILE, O_WRONLY);
  char* username = readline("Enter your username: ");
  char prompt[strlen(username) + 3];
  snprintf(prompt, sizeof(prompt), "%s: ", username);

  while (1) {
    char* message = readline(prompt);
    if (strncmp(message, "/exit", 5) == 0) {
      break;
    }

    Message msg;
    strncpy(msg.username, username, sizeof(msg.username));
    strncpy(msg.message, message, sizeof(msg.message));
    time_t t = time(NULL);
    msg.timestamp = *localtime(&t);

    write(fd, (char*)&msg, sizeof(Message));
  }

  close(fd);

  return 0;
}
```

```c
#include <fcntl.h>
#include <readline/readline.h>
#include <stdio.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

#define FIFO_FILE "fifo_file"
#define MAX_SIZE 100

// In actual applications, this must be (de-)serialized
typedef struct message {
  char username[MAX_SIZE];
  char message[MAX_SIZE];
  struct tm timestamp;
} Message;
```

Reading/Writing to Pipes

- **Prematurely Closing Pipes**
  - When writing to pipes that do not have an open reading end, the kernel sends **`SIGPIPE`** to the writing process.
    - » When ignored, **`write()`** returns **`-1`** and sets **`ERRNO`** to **`EPIPE.`**
  - When reading from pipes that do not have an open writing end, the **`read()`** function returns **`0`** (end-of file).
- **PIPE_BUF**
  - Maximum amount of data that can be atomically written to a pipe (without being interleaved among multiple writers)
- **Applies to both anonymous and named pipes**

Lab Exercise

- **Make *mini shell* that supports I/O redirection & pipes**
  - Should be under an infinite loop with **conditional exit**
  - Should execute the entered command on the **child process**
  - Should make the parent process **wait** for all the child processes to finish before exiting
  - Should execute programs in **$PATH**
  - Should handle **3 redirections(<,>, >>)** and **pipe(|)**
  - (Basically, week5 exercise + redirection/pipe)

- **No restrictions on header files**

Lab Exercise

- **There will be only one redirection/pipe in a line**
  - e.g. Commands like the example below will not be tested
    - » `cat < test.txt | grep test`
    - » That's for PA2
  - Hint: Since there is only one redirection/pipe, args can be divided into two when parsing.
    - » One for the (first) command
    - » The other for filename/second command
- **Skeleton code:**
  - You can use your submission for week5.

Lab Exercise

- **Example**

```
💻 ./w7
$ ls -1L
Makefile
w7
week7.c
$ echo 1 + 1 | bc
2
$ date > time.txt
$ cat < time.txt
Wed Apr 15 08:03:36 AM UTC 2026
$ rm time.txt w7
$ tar -czf ../2026000000_w7.tar.zst -C ../week7 .
$ ls ../2026000000_w7.tar.zst
../2026000000_w7.tar.zst
$ exit
```

Exercise submission

- **Submit your source code and Makefile in iCampus**
  - Makefile should generate ./w7 executable
  - Bundle *source code* and *Makefile* with tar command

    ```
    $ tar caf [student_id]_w7.tar.zst week7
    ```

  - We'll grade your submission with **make**
    - » If compilation fails, your points for this exercise will be zero

- **No class next week as it is midterms week**